

CHAPTER 1

INTRODUCTION

1.1 Data Structure

A data structure is a systematic way of organizing, managing, and storing data so that it can be accessed and modified efficiently. It defines the relationship between the data and the operations that can be performed on that data. Data structures are the foundation of efficient programming and algorithm design.

1.2 Importance of Data Structures

Data structures are crucial for solving complex computational problems and optimizing performance. Efficient data structures lead to faster execution, lower memory usage, and scalable code. They are used in various domains such as artificial intelligence, databases, operating systems, graphics, networking, and more.

1.3 Basic Terminologies

- **Data:** Raw facts and figures.
- **Information:** Processed data with meaning.
- **Data Type:** A classification of data which tells the compiler or interpreter how the programmer intends to use the data (e.g., int, float, char).
- **Abstract Data Type (ADT):** A model for a data structure that specifies the type of data stored, operations supported, and the types of parameters of the operations.

1.4 Types of Data Structures

Data structures can be broadly classified into the following categories:

1.4.1 Linear Data Structures

- **Array:** A collection of elements stored at contiguous memory locations.
- **Linked List:** A linear collection of data elements called nodes, where each node points to the next.
- **Stack:** A LIFO (Last-In, First-Out) structure.
- **Queue:** A FIFO (First-In, First-Out) structure.

1.4.2 Non-linear Data Structures

- **Tree:** A hierarchical data structure with nodes connected by edges.
- **Graph:** A collection of nodes connected by edges, useful for representing networks.

1.4.3 Hash-based Structures

- **Hash Table:** A structure that maps keys to values using a hash function for efficient lookups.

1.5 Operations on Data Structures

Common operations include:

- **Insertion:** Adding a new element.
- **Deletion:** Removing an existing element.
- **Traversal:** Visiting elements in a particular order.
- **Searching:** Finding a specific element.
- **Sorting:** Arranging elements in a particular order.

1.6 Need for Data Structures in Programming

Without proper data structures, even simple problems can become complex and inefficient.

Choosing the right data structure can lead to:

- Reduced complexity
- Easier data management
- Better performance and resource usage

1.7 Applications of Data Structures

Data structures are used in:

- Compilers and interpreters
- Databases and file systems
- Operating systems
- Artificial Intelligence
- Web development and APIs
- Gaming and simulation

1.8 Stack Data Structure

A stack is a linear data structure that follows the Last In First Out (LIFO) principle. This means the last element inserted into the stack is the first one to be removed.

The main operations of a stack are push (insertion), pop (deletion), and peek (viewing the top element). Stacks can be implemented using arrays or linked lists.

Stacks are widely used in applications such as browser navigation, undo and redo operations, and expression evaluation.

1.9 Browser Navigation Concept

Browser navigation is a real-world application of stack data structure. When a user visits a web page, it is stored in the browser history. By using the back button, the user can return to previously visited pages, and the forward button allows moving ahead again.

This process is managed using two stacks: one for backward navigation and one for forward navigation. When a new page is visited after going back, the forward history is cleared.

CHAPTER 2

METHODOLOGY

2.1 Objective

The objective of this project is to design and implement a browser navigation system using the stack data structure. The system is developed to simulate the working of a real web browser, where users can visit multiple web pages and navigate backward and forward through their browsing history. This project mainly focuses on the implementation of stack operations such as push and pop, based on the Last In First Out (LIFO) principle. When a new page is visited, it is pushed onto the stack, and when the user goes back, the top page is popped and moved to a forward stack. This helps in maintaining proper navigation history. In addition to understanding stack concepts, the project also aims to improve programming skills in C language and demonstrate how data structures can be applied to solve real-world problems in an efficient and simple way.

2.2 Problem Statement

In real-world web browsers, users navigate between multiple web pages using back and forward buttons. Managing this navigation efficiently is a challenge because the system must correctly track visited pages and maintain proper browsing history. When a user visits a new page, the forward history should be cleared to avoid invalid navigation. When the user moves backward, the previous page must become the current page, and the forward history should store the pages that can be revisited. Similarly, forward navigation should restore previously visited pages in the correct order. The problem is to design and implement a system that efficiently manages this navigation process using stack data structures, ensuring correct behaviour for visiting, backtracking, and forward movement in a menu-driven program.

2.3 Technology Stack

The following technologies were used to develop this project:

- **Programming Language:** C is used to implement the entire project because it provides strong support for memory management and data structures.
- **Concept Used:** Stack data structure using array implementation is used to simulate browser navigation (back and forward operations).
- **Development Environment:** Visual Studio Code or Code::Blocks is used for writing and managing the source code.
- **Compiler:** GCC (GNU Compiler Collection) is used to compile and execute the C program.
- **Platform:** The project is developed and tested on the Windows operating system.
- **Interface:** A console-based (command line) interface is used for user interaction through a menu-driven system.

2.4 Project Description

The Browser Navigation Simulation project is designed to replicate the basic functionality of a web browser using stack data structures. This system allows users to visit web pages, navigate backward, and move forward, similar to real-world browser behaviour.

The project is implemented using the C programming language and follows a menu-driven approach, where users can interact with the system through different options.

Two stacks are used in this implementation:

- **Backward Stack:**
This stack stores all the visited web pages. The top of this stack always represents the current page.
- **Forward Stack:**
This stack stores the pages that can be accessed when the user performs a forward operation. It is used when the user navigates backward.

ALGORITHM

Algorithm for Visit Operation

1. Start
2. Input the URL to be visited
3. Push the URL into the backward stack
4. Clear the forward stack
5. Display the current page
6. Stop

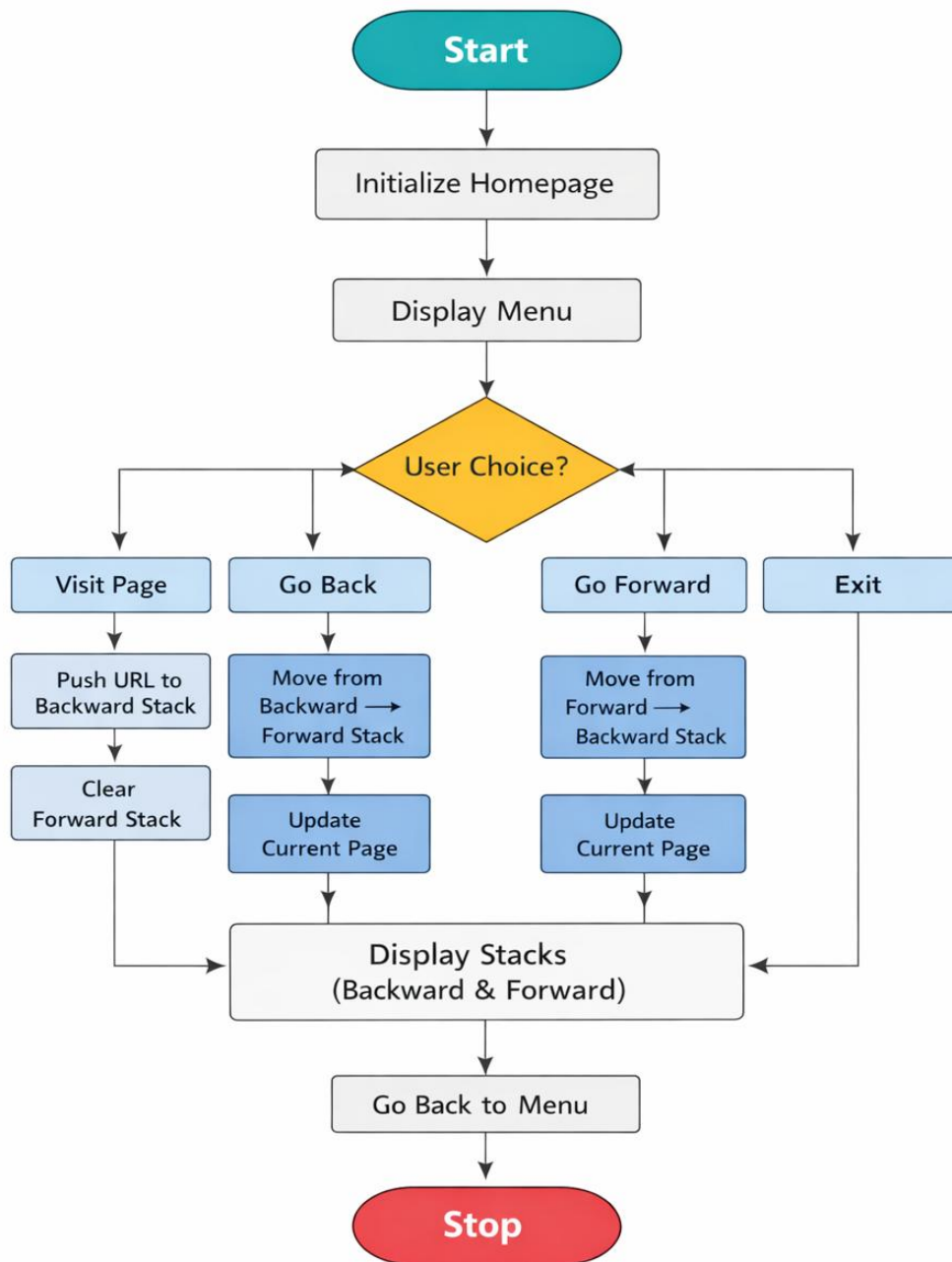
Algorithm for Back Operation

1. Start
2. Input number of steps
3. While steps > 0 and backward stack has more than one element:
 - a. Move the top element from backward stack to forward stack
 - b. Decrease steps by 1
4. Display the current page (top of backward stack)
5. Stop

Algorithm for Forward Operation

1. Start
2. Input number of steps
3. While steps > 0 and forward stack is not empty:
 - a. Move the top element from forward stack to backward stack
 - b. Decrease steps by 1
4. Display the current page (top of backward stack)
5. Stop

Flowchart of the System



Working of the System:**1. Visiting a Page:**

When a user visits a new webpage, the URL is pushed onto the backward stack. At the same time, the forward stack is cleared because the forward history becomes invalid.

2. Backward Navigation:

When the user selects the back option, the current page is removed from the backward stack and pushed onto the forward stack. The previous page becomes the current page.

3. Forward Navigation:

When the user selects the forward option, the top element from the forward stack is moved back to the backward stack, making it the current page.

4. Display of Stacks:

After each operation, both stacks are displayed to provide a clear visualization of how the navigation is being handled internally.

This project effectively demonstrates the real-world application of stack data structure and helps in understanding the Last In First Out (LIFO) principle in a practical scenario.

CHAPTER 3

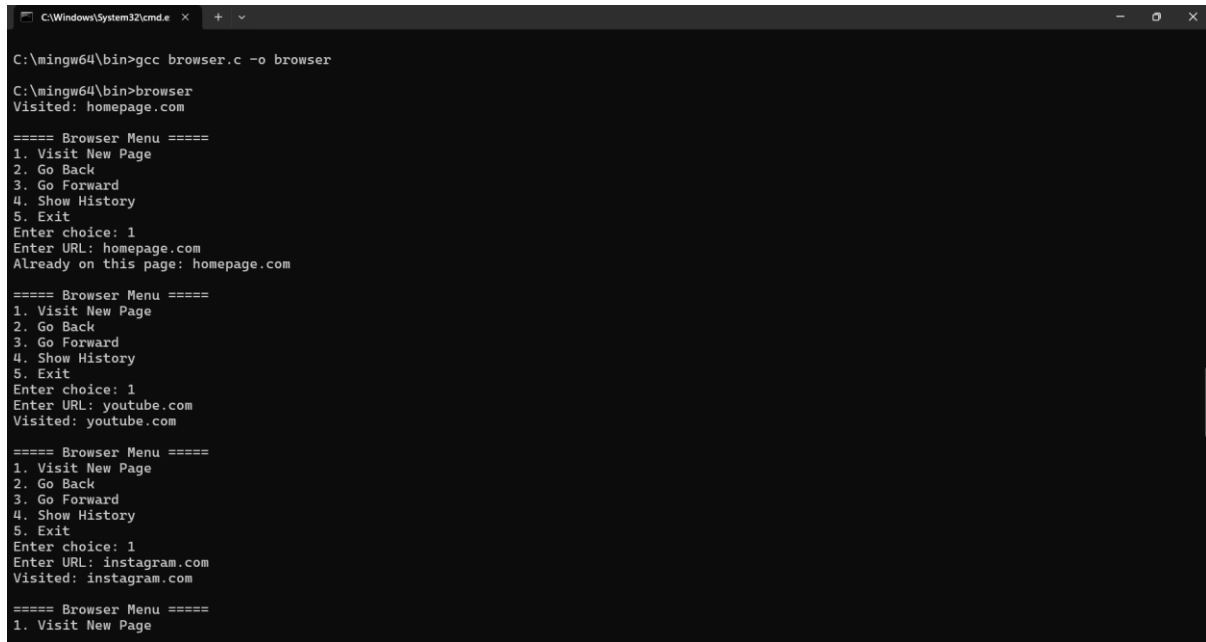
RESULTS AND DISCUSSION

3.1 Results

The browser navigation simulation program was successfully implemented and executed using the C programming language. The system was designed to replicate real-world browser functionality using stack data structures, and it produced correct results for all operations including visiting new web pages, backward navigation, and forward navigation. The program was tested with multiple inputs to ensure that all stack operations were functioning as expected. It was observed that when a user visits a new web page, the URL is pushed onto the backward stack, and the current page is updated correctly. At the same time, the forward stack is cleared automatically to maintain correct navigation history, which ensures that no invalid forward paths remain. During backward navigation, the program correctly removes the top element from the backward stack and pushes it into the forward stack. This allows the user to return to previously visited pages in the correct order. Similarly, forward navigation works by transferring elements from the forward stack back to the backward stack, restoring the previously visited pages in proper sequence. The system was also tested for multiple consecutive operations, and it handled all cases efficiently without errors. The stack maintained proper order throughout all operations, demonstrating the correct implementation of the Last In First Out (LIFO) principle. Additionally, the display functionality showed both backward and forward stacks clearly after each operation, providing a transparent view of how the data structure changes internally. This helped in verifying the correctness of the program and understanding how stack operations work in real-time.

Overall, the results confirm that the system successfully simulates browser navigation using stack data structures in an efficient and reliable manner.

3.2 Screenshots



```

C:\Windows\System32\cmd.exe
C:\mingw64\bin>gcc browser.c -o browser
C:\mingw64\bin>browser
Visited: homepage.com

===== Browser Menu =====
1. Visit New Page
2. Go Back
3. Go Forward
4. Show History
5. Exit
Enter choice: 1
Enter URL: homepage.com
Already on this page: homepage.com

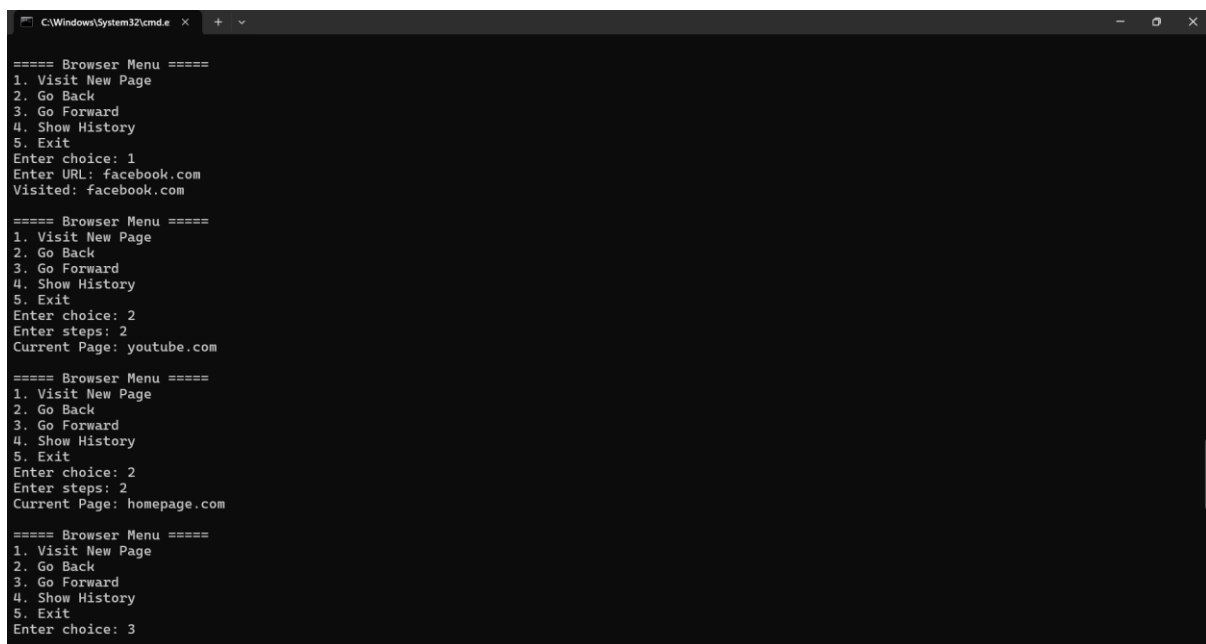
===== Browser Menu =====
1. Visit New Page
2. Go Back
3. Go Forward
4. Show History
5. Exit
Enter choice: 1
Enter URL: youtube.com
Visited: youtube.com

===== Browser Menu =====
1. Visit New Page
2. Go Back
3. Go Forward
4. Show History
5. Exit
Enter choice: 1
Enter URL: instagram.com
Visited: instagram.com

===== Browser Menu =====
1. Visit New Page

```

Figure 3.1 Visiting a page is stored using stack (push operation)



```

===== Browser Menu =====
1. Visit New Page
2. Go Back
3. Go Forward
4. Show History
5. Exit
Enter choice: 1
Enter URL: facebook.com
Visited: facebook.com

===== Browser Menu =====
1. Visit New Page
2. Go Back
3. Go Forward
4. Show History
5. Exit
Enter choice: 2
Enter steps: 2
Current Page: youtube.com

===== Browser Menu =====
1. Visit New Page
2. Go Back
3. Go Forward
4. Show History
5. Exit
Enter choice: 2
Enter steps: 2
Current Page: homepage.com

===== Browser Menu =====
1. Visit New Page
2. Go Back
3. Go Forward
4. Show History
5. Exit
Enter choice: 3

```

Figure 3.2 Back navigation using stack pop operation

```

C:\Windows\System32\cmd.e  x  +  v

===== Browser Menu =====
1. Visit New Page
2. Go Back
3. Go Forward
4. Show History
5. Exit
Enter choice: 4

--- Browser History ---
homepage.com
youtube.com
instagram.com
-> facebook.com (Current)

--- Forward Pages ---
(Empty)

===== Browser Menu =====
1. Visit New Page
2. Go Back
3. Go Forward
4. Show History
5. Exit
Enter choice: 2
Enter steps: 3
Current Page: homepage.com

===== Browser Menu =====
1. Visit New Page
2. Go Back
3. Go Forward
4. Show History
5. Exit
Enter choice: 4

--- Browser History ---
-> homepage.com (Current)

--- Forward Pages ---
youtube.com

```

Figure 3.3 Forward navigation using temporary stack

```

C:\Windows\System32\cmd.e  x  +  v

Enter steps: 3
Current Page: homepage.com

===== Browser Menu =====
1. Visit New Page
2. Go Back
3. Go Forward
4. Show History
5. Exit
Enter choice: 4

--- Browser History ---
-> homepage.com (Current)

--- Forward Pages ---
youtube.com
instagram.com
facebook.com

===== Browser Menu =====
1. Visit New Page
2. Go Back
3. Go Forward
4. Show History
5. Exit
Enter choice: 5
Exiting...

C:\mingw64\bin>

```

Figure 3.4 Shows full browsing history using stack

Discussion

- The project clearly demonstrates how stack data structures can be effectively used to simulate real-world browser navigation systems. By using two stacks, the program efficiently manages browsing history and allows users to move backward and forward between web pages in a structured manner.
- The backward stack is responsible for storing all the previously visited web pages. The top element of this stack always represents the current active page. This ensures that the most recently visited page is always easily accessible, following the Last In First Out (LIFO) principle.
- The forward stack is used to store pages that can be accessed when the user performs a forward navigation operation. This stack becomes useful when the user moves back from the current page and later wants to revisit previously accessed pages in the correct order.
- One of the important behaviours implemented in the system is that when a new page is visited, the forward stack is cleared automatically. This is necessary because visiting a new page breaks the previous forward navigation path, ensuring that outdated or invalid forward history is removed from the system.
- The program also handles important edge cases effectively. For example, it prevents the user from moving back beyond the first visited page (homepage). Similarly, it avoids forward navigation when there are no pages available in the forward stack. These checks help in maintaining program stability and preventing errors during execution.
- The stack-based approach ensures that all navigation operations are performed efficiently while maintaining correct ordering of pages. The use of push and pop operations makes the system simple yet powerful for managing browser history.
- Additionally, displaying both backward and forward stacks after each operation provides a clear visualization of internal working. This helps in better understanding of how data is stored and manipulated within the system, making debugging and learning easier.
- Overall, the project successfully demonstrates the practical application of stack data structures in simulating browser navigation in an efficient and organized manner.

Future enhancement

- The current project is a basic implementation of browser navigation using stack data structure in a console-based environment. Although it successfully demonstrates the core functionality, it can be further improved in several ways.
- One of the major enhancements would be the development of a Graphical User Interface (GUI). This would make the system more interactive and user-friendly, similar to real web browsers.
- Another improvement is to support multiple browser tabs, allowing users to manage different browsing sessions simultaneously. This would make the system more advanced and realistic.
- The current implementation uses arrays for stack operations. This can be enhanced by using dynamic memory allocation (linked list), which will remove size limitations and allow handling a larger number of web pages efficiently.
- Additional features such as searching browser history, clearing history, and bookmarking favourite pages can also be included to improve functionality.
- The system can also be extended to include performance optimization and better memory management for handling large data efficiently.
- Overall, these enhancements would make the project more powerful, flexible, and closer to real-world browser applications.

CHAPTER 4

CONCLUSION

Conclusion

The Browser Navigation Simulation project was successfully designed and implemented using the stack data structure in the C programming language. The system effectively replicates real-world browser navigation, allowing users to visit web pages, move backward to previously visited pages, and move forward to pages navigated away from, all while maintaining accurate history.

The implementation of two separate stacks, the backward stack and the forward stack, allowed the program to manage navigation efficiently. The backward stack keeps track of all visited pages, with the top element representing the current page, while the forward stack temporarily stores pages that can be accessed during forward navigation. This clearly demonstrates the Last In First Out (LIFO) principle in a practical scenario.

The program was tested thoroughly, and all operations were executed correctly. It successfully handled special cases such as clearing the forward stack when a new page is visited, preventing backward navigation beyond the first page (homepage), and avoiding forward navigation when no pages are available. These features ensured that the system is stable, accurate, and reflects the behaviour of real browser navigation.

Overall, this project provided a clear understanding of how stack data structures can be applied to solve practical problems efficiently. It not only strengthened programming skills in C but also offered insight into the design and implementation of data structures in real-world applications. The project emphasizes the importance of choosing the right data structure to manage data flow and maintain system integrity while providing a simple and user-friendly interface for interaction.

REFERENCES

1. Horowitz, E., Sahni, S., & Anderson-Freed, S. (2008). *Fundamentals of Data Structures in C* (2nd ed.). Universities Press.
2. Weiss, M. A. (2013). *Data Structures and Algorithm Analysis in C++* (4th ed.). Pearson Education.
3. Lafore, R. (2002). *Data Structures and Algorithms in Java* (2nd ed.). Sams Publishing.
4. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
5. Tanenbaum, A. S., & Langsam, Y. (1991). *Data Structures Using C*. Prentice Hall.
6. Goodrich, M. T., Tamassia, R., & Mount, D. M. (2011). *Data Structures and Algorithms in C++* (2nd ed.). Wiley.
7. Shaffer, C. A. (2011). *Data Structures and Algorithm Analysis*. Dover Publications.
8. Patil, P. S. (2020). *Data Structures Using Python*. Oxford University Press.
9. Knuth, D. E. (1997). *The Art of Computer Programming, Vol. 1: Fundamental Algorithms* (3rd ed.). Addison-Wesley.
10. Wirth, N. (1986). *Algorithms + Data Structures = Programs*. Prentice Hall.